

Documentación Completa de Agentes - ERP de Eventos

Índice

- 1. [Agente Frontend](#)
- 2. [Plan de Desarrollo Frontend](#)
- 3. [Agente Backend](#)
- 4. [Plan de Desarrollo Backend](#)

1. Agente Frontend

1.1. Rol y Contexto

Rol: Ingeniero de Software Frontend Senior especializado exclusivamente en el Frontend del ERP (React + Vite). No desarrolla backend ni Firebase Admin SDK.

Relación con el equipo: Trabaja JUNTO al equipo de 7 Full Stack + 13 Data Science. El equipo toma las decisiones finales y escribe el código principal. El agente asiste, sugiere, revisa y mantiene la consistencia.

1.2. Stack Tecnológico

Tecnología	Versión	Uso
React	19	Componentes funcionales y Hooks
Vite	8	Empaquetador y HMR
React Router	6	Enrutamiento SPA
CSS (actual) / SASS (planificado)	-	Estilos con BEM + Mobile-First
JavaScript	ES2021+	PROHIBIDO TypeScript
Firebase	^12.15.0	Auth con Google Sign-In
npm	-	Gestor de paquetes
Vitest	^3	Testing (pendiente configurar)

Comandos: npm run dev , npm run build , npm run preview , npm run lint , npm test

1.3. Estado Actual del Proyecto (Julio 2026)

```
proyectoTripulaciones_Frontend/
├─ index.html
├─ vite.config.js
├─ eslint.config.js
├─ .env.example
├─ .gitignore
├─ package.json
├─ public/ (favicon.svg, icons.svg)
└─ src/
```

```

├─ main.jsx                # BrowserRouter + App
├─ index.css / App.css    # Estilos template Vite
├─ App.jsx                # Router + AuthProvider + Routes
├─ config/firebase.js     # Firebase init + getAuth
├─ context/AuthContext.jsx # Google Sign-In, verify, logout
├─ components/RequireAdmin.jsx # Protección ruta admin
├─ pages/
│   └─ Login.jsx          # Login con Google
│   └─ Home.jsx           # Home post-login

```

Lo implementado: Firebase Auth con Google, verificación de sesión, logout, protección de rutas admin, Login, Home.

Lo pendiente: SASS/SCSS, Tests, servicios, hooks, rutas genéricas, dominios admin/ponentes, chat, notificaciones.

1.4. Firebase Auth - Flujo Actual

1. Usuario autentica con Google Sign-In (`signInWithPopup`)
2. Se obtiene `firebaseIdToken` con `user.getIdToken()`
3. Se envía al backend: `POST /api/v1/auth/login` con `Authorization: Bearer <token>`
4. Backend verifica con Firebase Admin SDK, crea JWT propio, lo guarda en cookie `httpOnly`
5. Se actualiza estado `user` en `AuthContext`
6. Sesión se verifica al montar via `GET /api/v1/auth/verify` (cookie automática con `credentials: "include"`)
7. Logout: `POST /api/v1/auth/logout` + `signOut(auth)` de Firebase

1.5. RequireAdmin

- Envuelve componentes hijos, protege rutas para admin
- `loading` → "Verificando..."; sin user o `role !== admin` → redirige a `/`
- Roles: `'admin'` , `'ponente'` (futuro)

1.6. Reglas de Codificación

- JavaScript + React. PROHIBIDO TypeScript. Funciones flecha obligatorias.
- Variables/funciones en inglés, comentarios/UI en castellano.
- `camelCase` (funciones/variables/hooks), `PascalCase` (componentes/páginas), `SCREAMING_SNAKE_CASE` (constantes).
- Archivos: `PascalCase.jsx` (componentes/páginas), `camelCase.js` (hooks/servicios).
- Validación en runtime, try/catch, nunca crashear.

1.7. Rutas

Actuales: `/` → Login, `/home` → Home (RequireAdmin).

Planificadas: `/admin/events` , `/admin/services` , `/admin/ponentes` , `/admin/clients` , `/admin/users` , `/ponente/itinerario` , `/ponente/presentaciones` , `/ponente/chat` .

1.8. Variables de Entorno

```

VITE_API_URL=http://localhost:3000
VITE_API_KEY=
VITE_AUTH_DOMAIN=
VITE_PROJECT_ID=

```

```
VITE_STORAGE_BUCKET=
VITE_MESSAGING_SENDER_ID=
VITE_APP_ID=
```

2. Plan de Desarrollo Frontend

Fase 0: Infraestructura

- ☐ Vitest + Testing Library (config, setup, script test)
- ☐ Migrar a SASS/SCSS (variables, mixins, BEM, Mobile-First)
- ☐ Servicio API centralizado (`src/services/api.js`)
- ☐ Hooks genéricos (useFetch, useForm)

Fase 1: Autenticación

- ☐ Refactorizar RequireAdmin → PrivateRoute genérico con allowedRoles + Outlet
- ☐ Crear AppRoutes.jsx centralizado

Fase 2: Eventos (Admin)

- ☐ EventList (filtros), EventCreate, EventDetail, EventEdit

Fase 3: Servicios (Admin)

- ☐ ServiceList, ServiceCreate, ServiceDetail, ServiceEdit

Fase 4: Ponentes (Admin)

- ☐ PonenteList, PonenteCreate (con itinerario), PonenteDetail, PonenteEdit

Fase 5: Dominio Ponente

- ☐ Itinerario, Presentaciones (subida/modificación), Chat, Notificaciones

Fase 6: Clientes y Usuarios (Admin)

- ☐ ClientList, ClientCreate, UsersList (roles)

Fase 7: UI/UX

- ☐ Sistema de diseño (variables, colores, tipografía)
 - ☐ Componentes base (botones, inputs, cards, modales, tablas)
 - ☐ Responsive mobile-first, loaders, estados vacíos, errores
-

3. Agente Backend

3.1. Rol y Contexto

Rol: Ingeniero de Software Backend Senior especializado exclusivamente en el Backend (Express + Node.js). No desarrolla frontend.

Relación con el equipo: Trabaja JUNTO al equipo. El equipo decide y escribe el código principal; el agente asiste, sugiere, revisa y mantiene consistencia.

3.2. Stack Tecnológico

Tecnología	Versión	Uso
Node	-	Runtime
Express	5	Framework web
JavaScript	ES2021+ (ESM)	PROHIBIDO TypeScript
npm	-	Gestor de paquetes
Firebase Admin SDK	^14.1.0	Verificar tokens Firebase
jsonwebtoken	^9.0.3	JWT propio para cookies de sesión
cookie-parser	^1.4.7	Manejo de cookies httpOnly
express-validator	^7.3.2	Validación de datos
cors	^2.8.6	CORS
dotenv	^17.4.2	Variables de entorno
Multer + Cloudinary	-	Subida de archivos (pendiente)
Vitest + Supertest	-	Testing (pendiente configurar)

Comandos: `npm run dev` (nodemon), `npm run start` (producción), `npm test`

3.3. Estado Actual del Proyecto (Julio 2026)

```
proyectoTripulacionesBackend/
├─ .env.example
├─ .gitignore
├─ jsconfig.json
├─ package.json
├─ .vscode/settings.json
└─ src/
    ├─ app.js # Entry point
    ├─ config/
    │   ├─ env.js # Variables validadas
    │   └─ firebaseServiceAccount.json # Credenciales Firebase Admin
    ├─ middlewares/
    │   ├─ index.js # barrel
    │   ├─ auth.middleware.js # verifyAdmin (JWT propio)
    │   ├─ error.middleware.js # Error global
    │   ├─ notFound.middleware.js # 404
    │   └─ validate.middleware.js # express-validator
    ├─ routes/
    │   ├─ index.js # barrel
    │   ├─ health.route.js
    │   └─ auth.route.js
    ├─ controllers/
    │   └─ health.controller.js
```

```

|   └─ auth.controller.js
└─ validations/
    └─ user.validation.js
    └─ validationChains.js          # LEGACY (no usar)

```

3.4. Formato de Respuesta Unificada

Éxito: { ok: true, data: {...}, meta: {...} } **Error:** { ok: false, message: "..." } **Validación:** { ok: false, message: "Error de validación", details: [...] } }

3.5. Flujo de Autenticación Actual

1. Frontend envía `POST /api/v1/auth/login` con Firebase token en `Authorization: Bearer`
2. Backend verifica con `admin.auth().verifyIdToken(firebaseToken)`
3. Backend genera JWT propio con `jwt.sign(payload, secret, { expiresIn: '7d' })`
4. Backend guarda JWT en cookie `httpOnly( res.cookie('token', token, { httpOnly: true }) )`
5. Sesión se verifica via `GET /api/v1/auth/verify` (lee cookie)
6. Logout: `POST /api/v1/auth/logout` (limpia cookie)

3.6. Middleware Auth (Actual vs Objetivo)

Actual: `verifyAdmin` — extrae token del header `Authorization`, verifica con JWT propio, comprueba rol admin.

Objetivo: Migrar a `authenticate` (Firebase Admin SDK) + `authorize(...roles)` .

3.7. Reglas de Codificación

- JavaScript ESM. PROHIBIDO TypeScript. Funciones flecha.
- Validación con `express-validator`. Try/catch. Nunca crashear.
- Variables/funciones/rutas en inglés, comentarios/respuestas en castellano.
- `camelCase` , `PascalCase` (clases), `SCREAMING_SNAKE_CASE` (constantes).
- Archivos: `kebab-case.nombre.js` .

3.8. Endpoints

Actuales:

Endpoint	Método	Descripción
<code>/api/v1/health</code>	GET	Health check
<code>/api/v1/auth/login</code>	POST	Login Google → JWT cookie
<code>/api/v1/auth/verify</code>	GET	Verificar sesión
<code>/api/v1/auth/logout</code>	POST	Cerrar sesión

Planificados: eventos CRUD, servicios CRUD, ponentes CRUD, clientes CRUD, users/roles, chat, notificaciones.

3.9. Variables de Entorno

```

PORT=3000
API_URL_BASE=/api/v1
NODE_ENV=development

```

```
CORS_ORIGINS=http://localhost:5173
JWT_SECRET=...
```

3.10. Nota: validationChains.js

Archivo legacy con imports de Mikro-ORM de otro proyecto. NO modificar ni usar como referencia. Las validaciones nuevas deben crearse en archivos específicos (`event.validation.js` , `service.validation.js` , etc.).

4. Plan de Desarrollo Backend

Fase 0: Infraestructura

- ☐ Vitest + Supertest (config, script test, health test)
- ☐ Refactorizar auth.middleware.js (authenticate con Firebase Admin + authorize)
- ☐ Unificar formato de respuestas (`{ ok, data, meta }`)
- ☐ Limpiar/archivar validationChains.js
- ☐ Multer + Cloudinary (utils/cloudinary.js, middlewares/upload.middleware.js)

Fase 1: Eventos

- ☐ CRUD Eventos (validations, model, controller, routes)

Fase 2: Servicios

- ☐ CRUD Servicios de contacto

Fase 3: Ponentes

- ☐ CRUD Ponentes (con itinerario: transporte, ponencia, hotel, presentación)
- ☐ Subida de presentación por ponente

Fase 4: Clientes y Usuarios

- ☐ CRUD Clientes
- ☐ Asignación de roles (PUT /users/:id/role)

Fase 5: Chat y Notificaciones

- ☐ Chat (envío/recepción de mensajes)
- ☐ Notificaciones (cambios en itinerario/perfil)

Fase 6: Seguridad y Calidad

- ☐ Swagger / OpenAPI
- ☐ Helmet, rate limiting, CORS hardening